

```
(d_env) project_d % | pip install -r requirements/development.txt
(d_env) project_d % | python manage.py makemigrations --settings
config.settings.development
(d_env) project_d % | python manage.py migrate --settings
config.settings.development
(d_env) project_d % | python manage.py createsuperuser --settings
config.settings.development
(d_env) project_d % | python manage.py runserver --settings
config.settings.development
```

Production environment

Here is a list of frequently used commands for the production environment. The key differences with the list above are the requirement file path and the way of running the Django app. As you cannot use the `runserver` command in the production environment, you need to use `systemctl` commands.

```
ubuntu@ip-xx:~/project_d$ | rm -r d_env
ubuntu@ip-xx:~/project_d$ | python3 -m venv d_env
ubuntu@ip-xx:~/project_d$ | source d_env/bin/activate
```

```
(d_env) ubuntu@ip-xx:~/project_d$ |python -m pip install --
upgrade pip pip install -r requirements/production.txt
(d_env) ubuntu@ip-xx:~/project_d$ | python manage.py
makemigrations
(d_env) ubuntu@ip-xx:~/project_d$ | python manage.py migrate
(d_env) ubuntu@ip-xx:~/project_d$ | python manage.py
createsuperuser
(d_env) ubuntu@ip-xx:~/project_d$ | python manage.py
collectstatic
(d_env) ubuntu@ip-xx:~/project_d$ | sudo systemctl restart
project_d
(d_env) ubuntu@ip-xx:~/project_d$ | systemctl status project_d
```